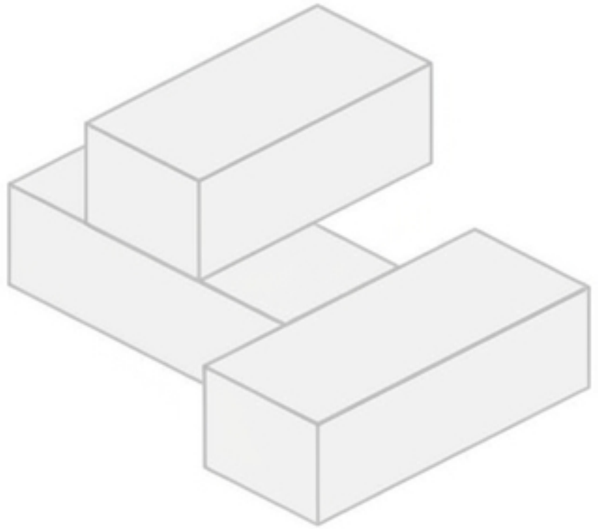
Dart Core Kütüphaneleri

Temel Yapı Taşları ve Mimari Rehber

Gemini Notebook

Dart

Sadece Flutter ile mobil geliştirme değil; aynı zamanda web ve sunucu (server-side) tarafında da çalışan, yüksek performanslı ve modern bir programlama dili.



Kütüphane (Library)

Yeniden kullanılabilir hazır sınıf ve fonksiyonlardan oluşan modüler yapı taşları.

```
Örnek: Matematiksel hesaplamalar için
tekerleği yeniden icat etmek yerine
dart:math kütüphanesini projeye dahil etmek.
```

Gemini Notebook

`dart:core`

Her programın kalbi. Dahili tipler, koleksiyonlar ve temel işlemler.

`dart:async`

Gelecekteki değerler (Future) ve asenkron olay akışları (Stream).

`dart:math`

Matematiksel sabitler (pi, e), trigonometri ve rastgele sayı üretimi.

`dart:convert`

Farklı veri formatları (JSON, UTF-8) arasında güvenli kodlama ve çözme.

`dart:io`

Native platformlar (Flutter, Sunucu, CLI) için dosya, soket ve HTTP giriş-çıkış işlemleri.

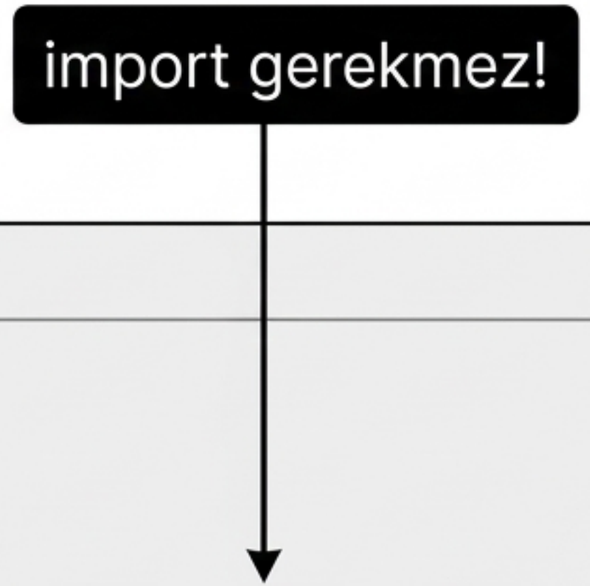
`dart:js_interop`

Web platformunda JavaScript kütüphaneleri ve DOM ile kusursuz entegrasyon.

Gemini Notebook

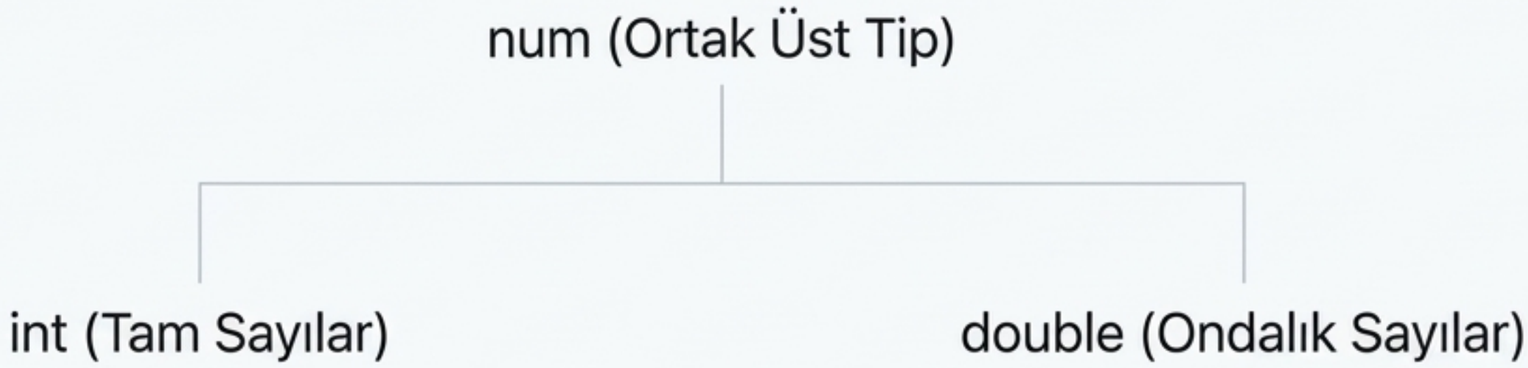
dart:core ve print()

Dart dilinin en kritik, yerleşik işlevlerini barındırır. Sayılar, metinler ve temel koleksiyonlar bu kütüphanenin içindedir. Her Dart programına otomatik olarak aktarıldığı için özel bir eklentiye ihtiyaç duymaz.




```
var name = 'Dart';
print('Merhaba $name'); // Top-level fonksiyon
```

Gemini Notebook



Kesin Dönüşüm

```
int.parse('42'); // 42
double.parse('3.14'); // 3.14

12.567.toStringAsFixed(2); // '12.57'
```

Güvenli Dönüşüm

tryParse: Hata fırlatmaz, null döner!

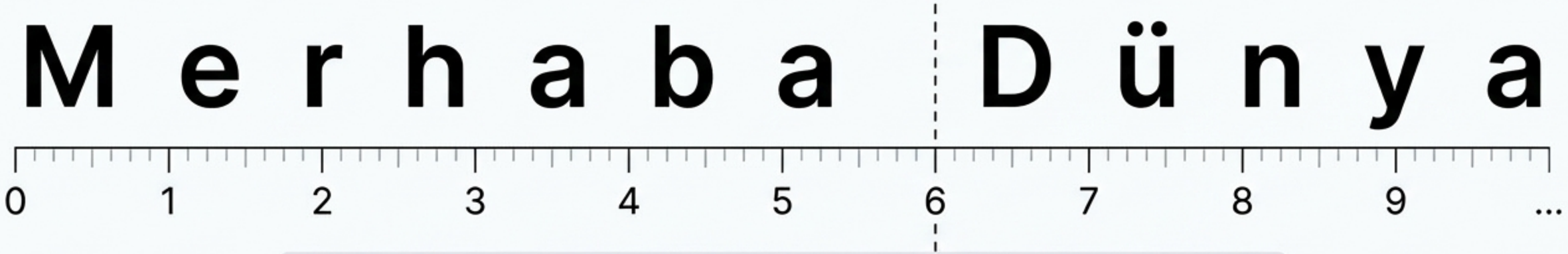
```
int.tryParse('42'); // 42
int.tryParse('abc'); // null
```

Gemini Notebook

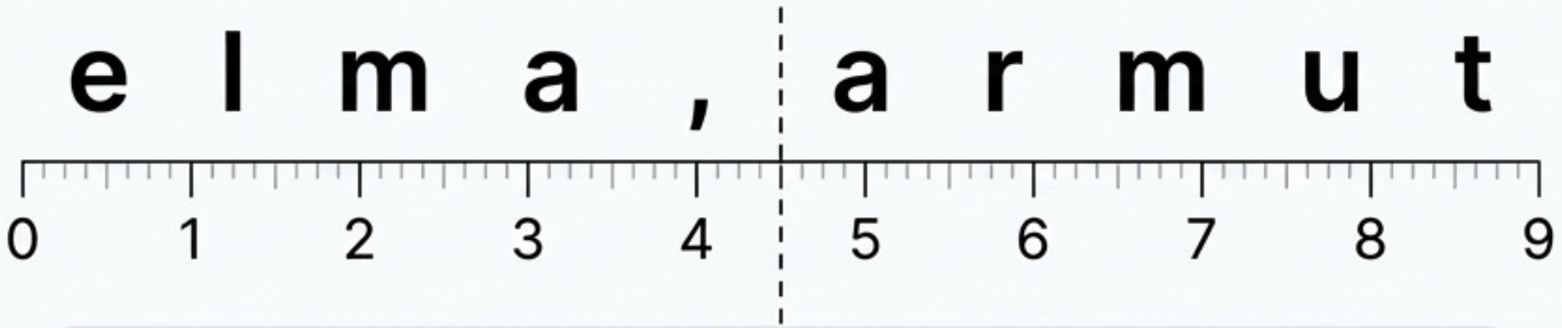
String: Değiştirilemez (Immutable) Metinler

String nesneleri yaratıldıktan sonra doğrudan değiştirilemez; her işlem yeni bir String üretir.

Arama Metotları: contains(), startsWith(), endsWith(), indexOf()

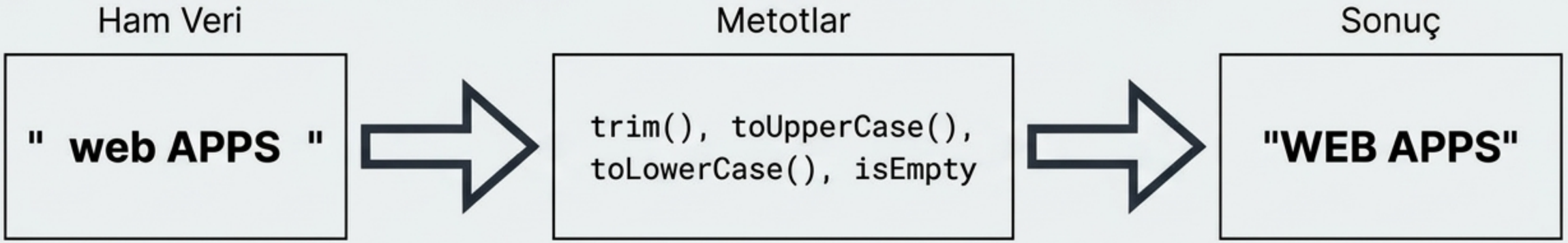



```
'Merhaba Dünya'.substring(0, 7); // 'Merhaba'
```




```
'elma,armut'.split(','); // ['elma', 'armut']
```

Gemini Notebook



Uyarı: Büyük/küçük harf dönüşümlerinde Türkçe karakterlere (i/İ, ı/I) dikkat edilmelidir. Çıktılar her dilin alfabesine uygun dönmeyebilir.

Düzenli İfadeler (RegExp)

```
final pattern = RegExp(r'\d+'); // Rakamları bul
print(pattern.hasMatch('Sipariş 123')); // true
```

Gemini Notebook

Koleksiyon Yapıları

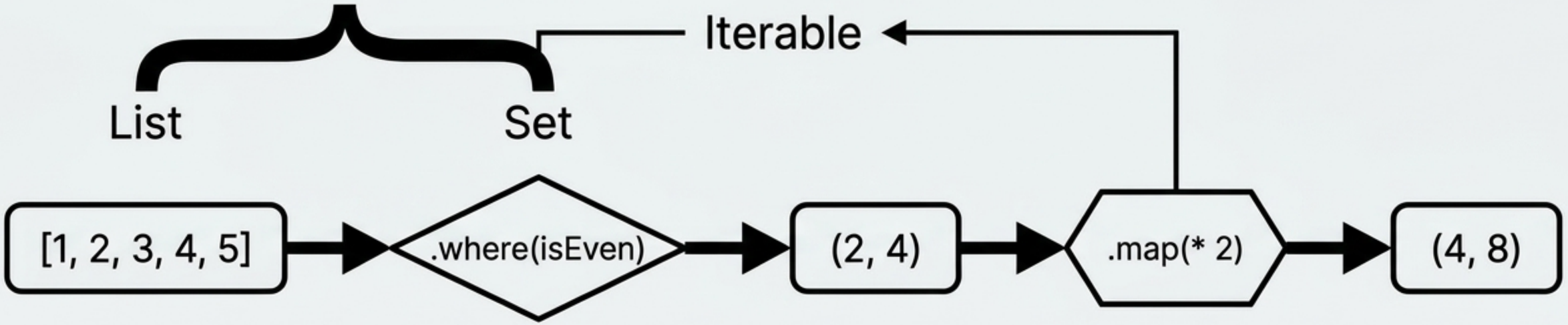
[• , • , •] List	{ • , * , + } Set	{ k : v } Map
Sıralı ve indeksli (0'dan başlar).	Benzersiz öğeler kümesi. Sırasızdır.	Anahtar–Değer (Key-Value) ilişkisi.
Aynı veriden birden fazla barındırabilen listeler.	Tekrar eden verilerin filtrelenmesi gerektiğinde.	Sözlük yapısı veya ID tabanlı hızlı veri aramaları.
`add(), removeAt(), indexOf(), length`	`add(), contains(), intersection()`	`containsKey(), putIfAbsent(), remove()`

Gemini Notebook

Koleksiyon İşlemleri ve Iterable

Iterable: List ve Set'in ortak üst yapısıdır. Elemanlara for-in döngüsü ile sırayla erişilmesini sağlar.

Diğer metotlar: forEach(), any(), every()

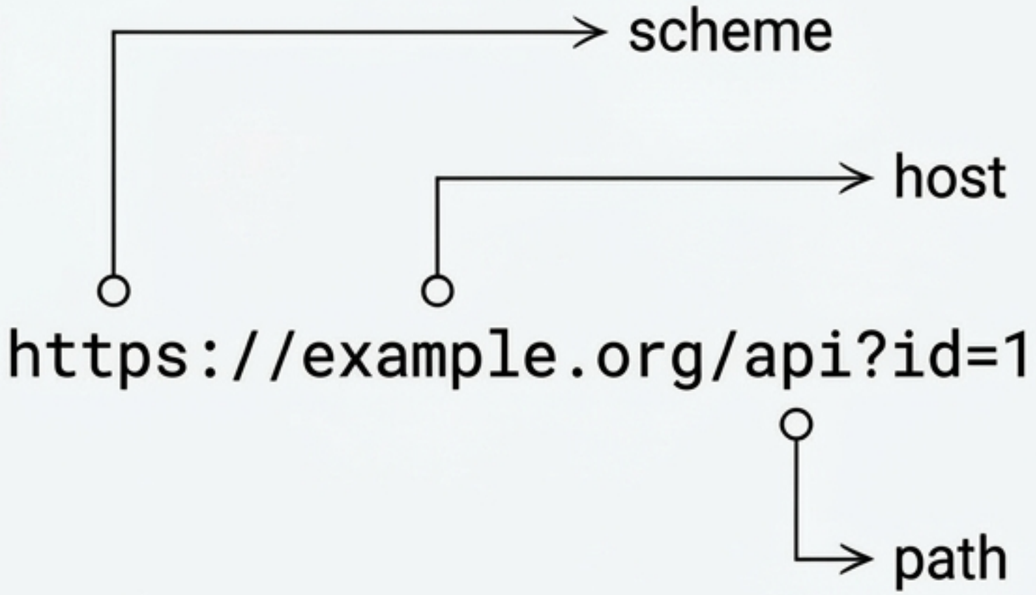



```
final numbers = [1, 2, 3, 4, 5];
final even = numbers.where((n) => n.isEven); // (2, 4)
final doubled = even.map((n) => n * 2);      // (4, 8)
```

Gemini Notebook

URI İşlemleri

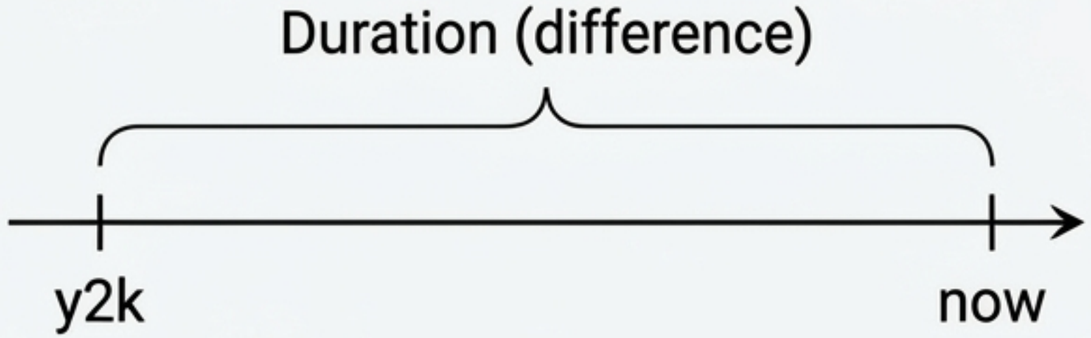
Özel karakterleri güvenle işler ve URL parçalarını ayrıştırır.




```
Uri.parse();
Uri.encodeComponent();
```

DateTime ve Duration

Zaman damgaları ve süre hesaplamaları.




```
var now = DateTime.now();
var y2k = DateTime.utc();
var y2k = DateTime.utc(2000);
var diff = now.difference(y2k);
print(diff.inDays);
```

Gemini Notebook



Mantıksal Eşitlik

Comparable arayüzü ile nesneleri sıralayın.
Metot: compareTo()
(Küçükse <0, eşitse 0, büyükse >0).
== operatörünü eziyorsanız, hashCode'u da ezmelisiniz.

Exception (İstisna)

Ele Alınabilir Durumlar. Beklenen ve programın çökmesini engellemek için **try-catch** blokları ile yönetilebilen sorunlardır.

```
try { // riskli işlem
} catch (e) { // yönet }
```

Error (Hata)

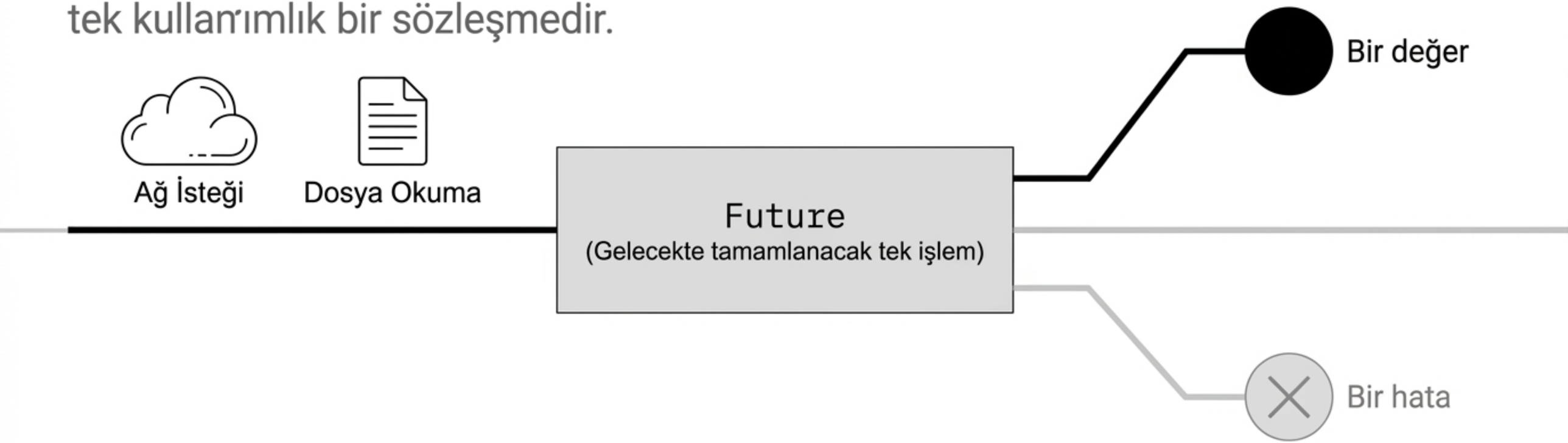
Programlama Hataları.
Genellikle yazılımcı kaynaklıdır (örn: ArgumentError).
Ele alınması (catch) tavsiye edilmez, kodun düzeltilmesi gerekir.

Gemini Notebook

Asenkron Programlama ve Future

Bekleme gerektiren işlemler anında sonuç vermez. Programın kilitlenmemesi için bu işlemler arka planda yürütülür.

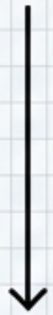
Future, bu asenkron işlemin nihai sonucunu temsil eden tek kullanımlık bir sözleşmedir.



Gemini Notebook

Future İşleme: async/await ve then()

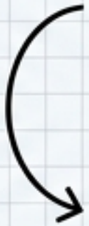
async / await (Modern & Doğrusal)



```
Future<void> main() async {
  final result = await getData();
  print(result);
}
```

Kod senkron gibi görünür, okuması kolaydır.

then() (Geleneksel)

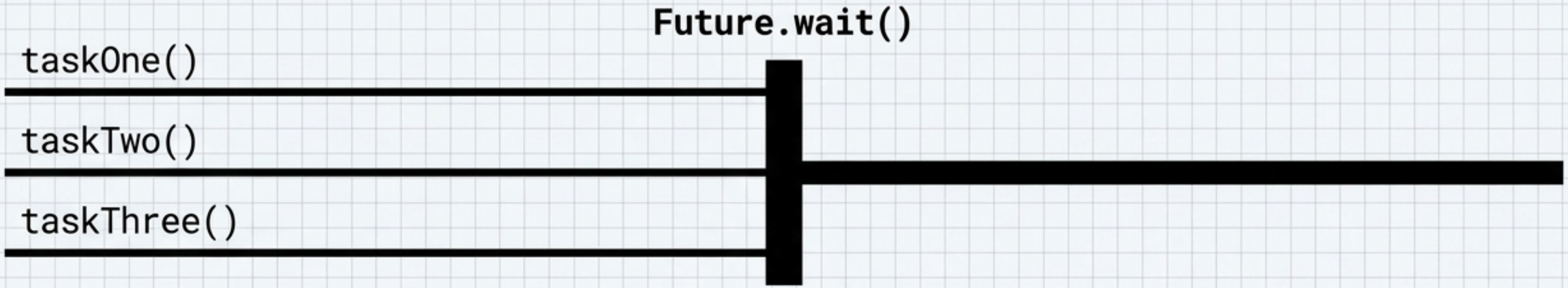


```
// then() ile
getData().then((value) {
  print(value);
});
```

Sonuç hazır olduğunda tetiklenecek bir fonksiyon kaydeder.

Gemini Notebook

Paralel İşlemler: Birden Fazla Future'ı Beklemek




```dart
await Future.wait([
  taskOne(),
  taskTwo(),
]);
```

Bağımsız asenkron işlemleri sırayla beklemek yerine, aynı anda başlatıp hepsinin tamamlanmasını tek bir noktada bekleyin.

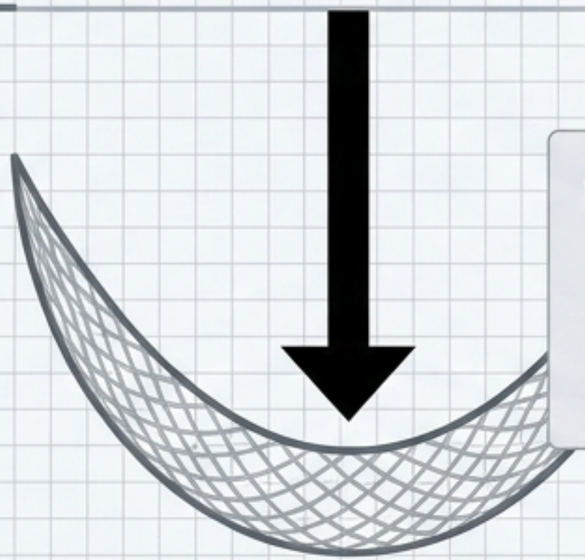
İşlemlerden herhangi biri başarısız olursa, **Future.wait()** anında **bir hata** fırlatır.

Gemini Notebook

Asenkron Hata Yönetimi

```
try {
  final data = await getData();
  print(data);
} catch (e) {
  print('Hata: $e');
}
```

async/await ile standart senkron hata yakalama mantığı.

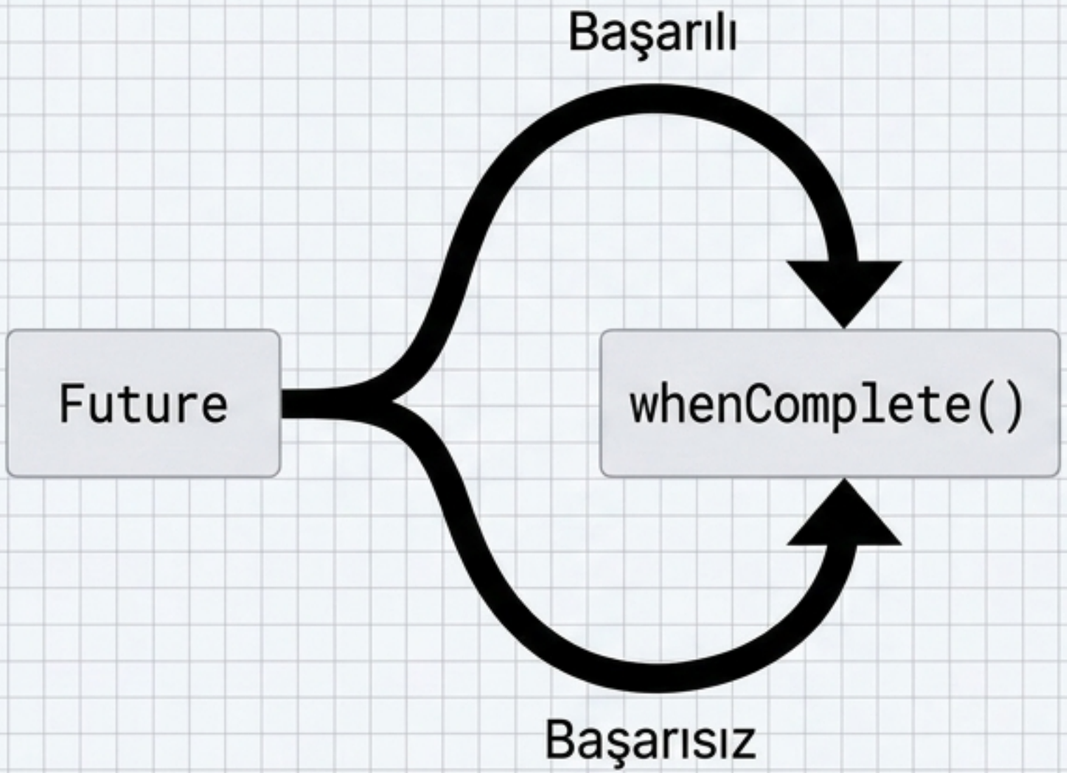


```
getData()
  .then(print)
  .catchError((e) {
  print('İşlem başarısız');
});
```

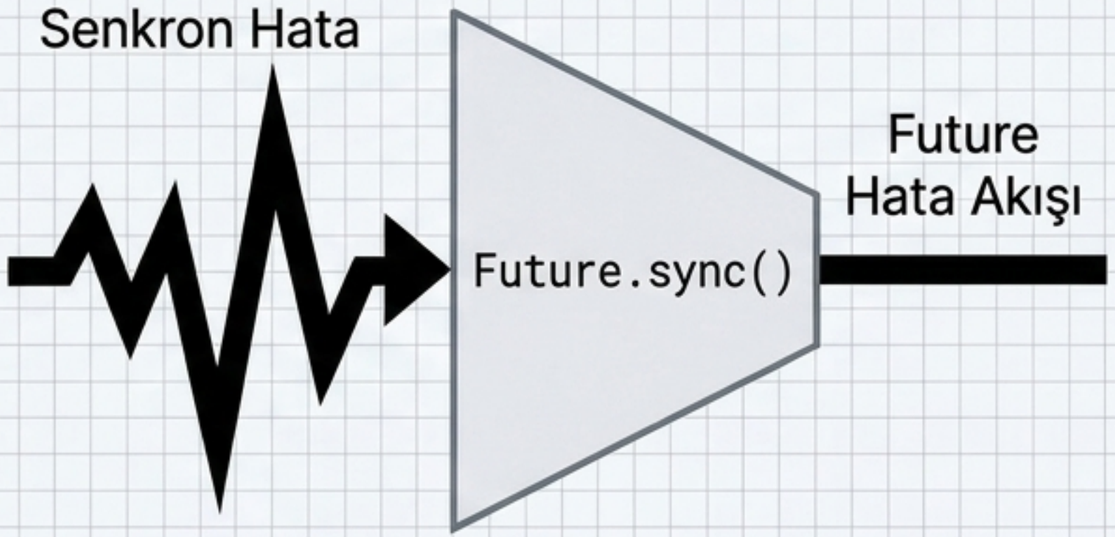
then() zincirinin sonunda fırlatılan hataları yakalayan güvenlik ağı.

Gemini Notebook

Yaşam Döngüsü ve Senkron Hatalar



İşlem başarılı da olsa, hata da verse en son çalışır (temizlik adımı).



Asenkron fonksiyonlardan dışarı sızabilecek senkron hataları güvenli Future akışına dahil eder.

Gemini Notebook

Stream Nedir? Future'dan Farkı

Future	Stream
	
Tek bir sonuç	Birden fazla olay
Bir kez tamamlanır	Zaman içinde ardışık veri iletir
`await` kullanılabilir	`await for` kullanılabilir

Future tek bir pakettir. **Stream** ise sürekli akan bir veri boru hattıdır.

Gemini Notebook

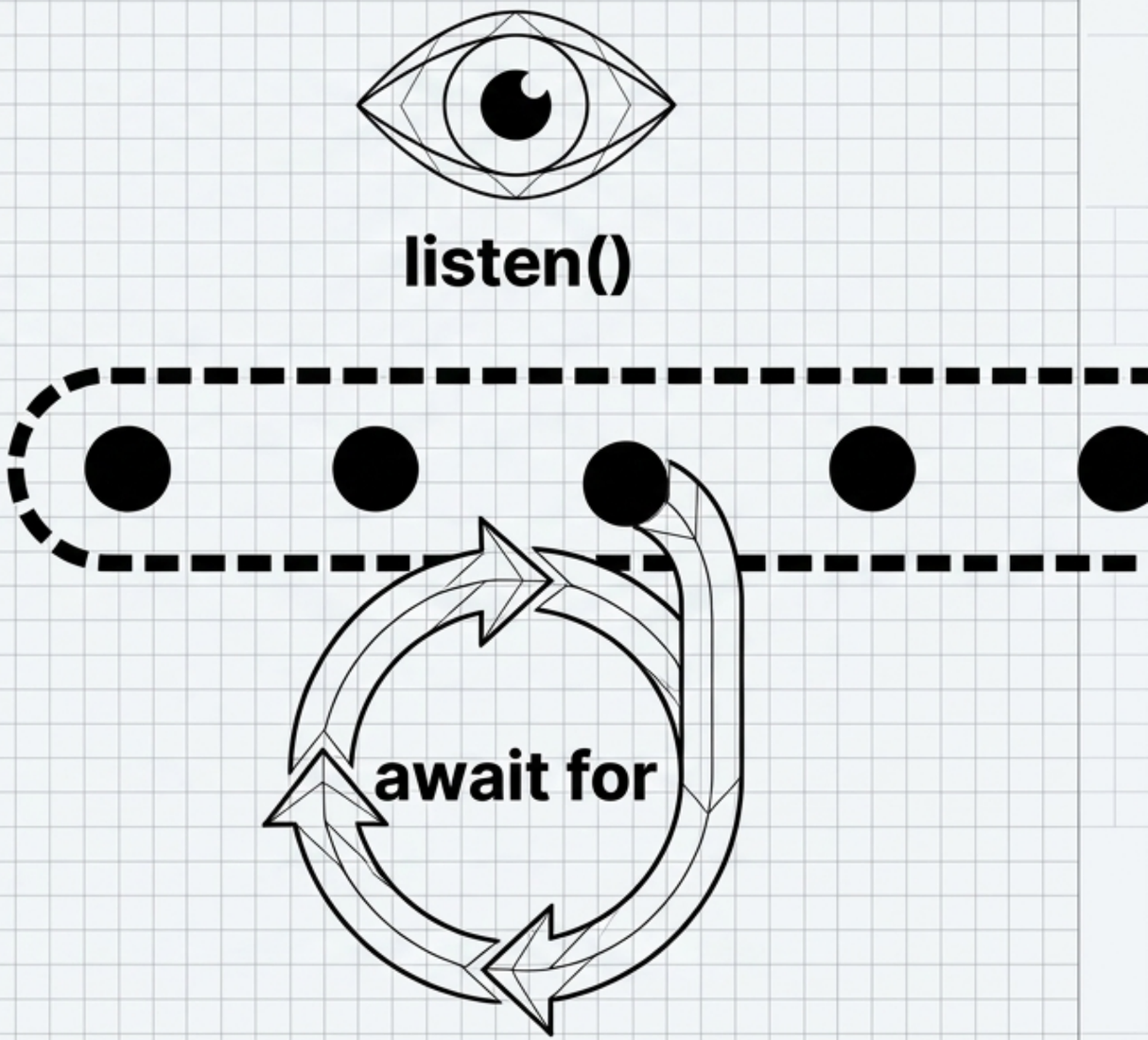
Stream’i Dinleme: listen() ve await for

```
stream.listen((value) {
  print(value);
});
```

Sürekli bir gözlemci kaydeder.

```
await for (final value in stream) {
  print(value);
}
```

Gelen verileri senkron bir döngü gibi sırayla işler.



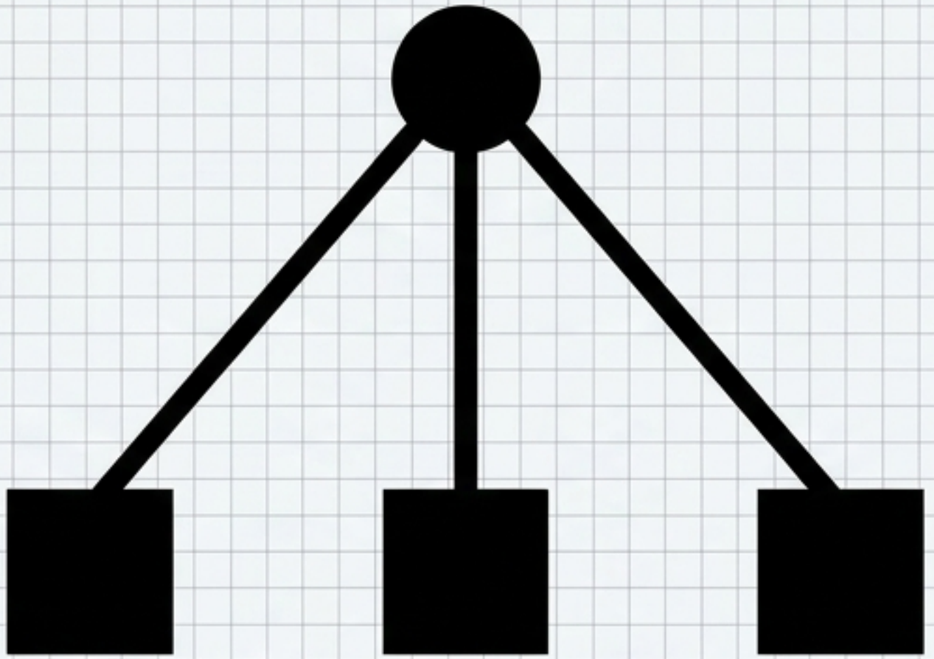
Gemini Notebook

Stream Türleri: Single vs Broadcast



`Single-subscription`

Tek dinleyici. Dosya okuma veya web isteği gibi verinin baştan sona tek bir yere aktarıldığı durumlar.

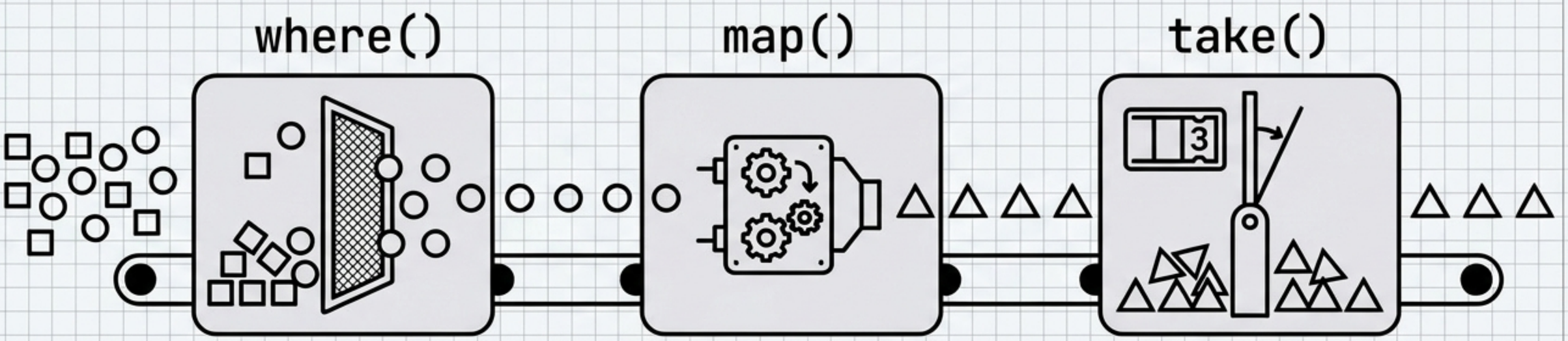


`Broadcast`

Birden fazla dinleyici. Fare tıklamaları veya bildirimler gibi, anlık olayları aynı anda birçok yere duyurmak için.

Gemini Notebook

Stream İşlemleri: Filtreleme ve Dönüştürme



Hata Yönetimi

`listen()` ile:

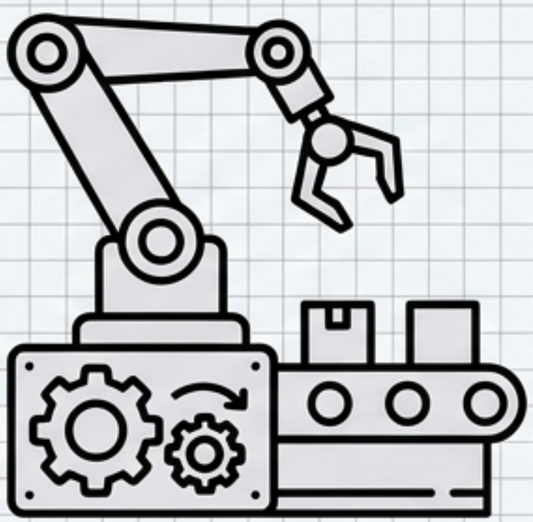
`onError`, `onDone` parametreleri

`await for` ile:

Geleneksel `try-catch` blokları

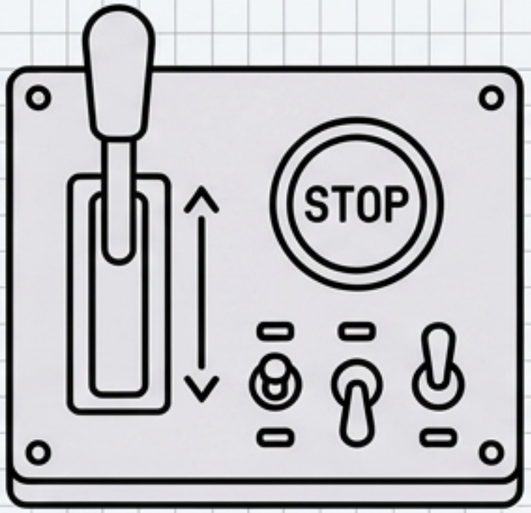
Gemini Notebook

Kendi Stream’imizi Oluşturma



`async*` ve `yield` (Otomatik Üretici)

```
Stream<int> numbers() async* {
  yield 1;
  yield 2;
  yield 3;
}
```



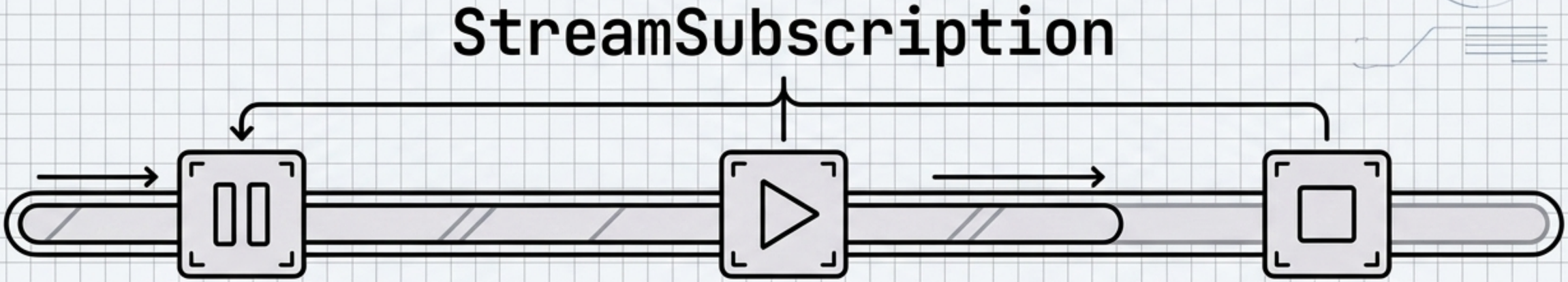
`StreamController` (Manuel Kontrol)

`add()` → Dışarıdan manuel olarak veri ekle.

`close()` → İşlem bitince makineyi güvenle kapat.

Gemini Notebook

Stream Aboneliği ve Yaşam Döngüsü



`pause()`

Akışı geçici olarak duraklat.

`resume()`

Durdurulan akışı kaldığı yerden devam ettir.

`cancel()`

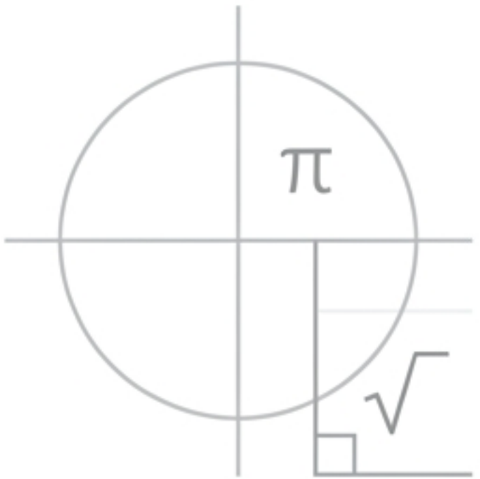
Aboneliği kalıcı olarak iptal et ve kaynakları serbest bırak.

Dinleme işlemi başladığında size bir abonelik nesnesi verilir. Arka plan sızıntılarını (memory leak) önlemek için işiniz bittiğinde aboneliği iptal etmek kritik önem taşır.

Gemini Notebook

`dart:math` — Hesaplama ve Rastgelelik

`dart:math` kütüphanesi, matematiksel sabitler ve karmaşık işlemler için optimize edilmiştir.





```dart
import 'dart:math';

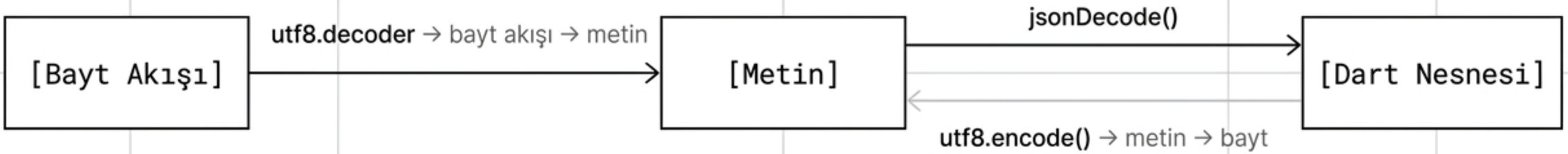
print(pi);
print(sin(pi / 6));
print(sqrt(25)); // 5.0
```

```dart
final random = Random();
print(random.nextInt(10)); // 0–9
```

Gemini Notebook

`dart:convert` — JSON ve UTF-8 Dönüşümleri

Farklı veri formatları arasında güvenli ve hızlı dönüşüm.

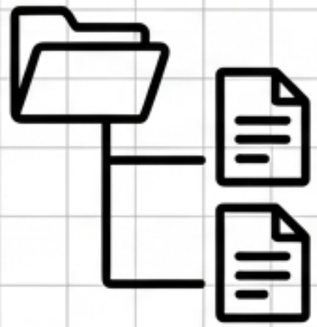



```dart
import 'dart:convert';

final jsonText = jsonEncode({'ad': 'Ali'});
final data = jsonDecode(jsonText);
```

Gemini Notebook

`dart:io` — Dosya İşlemleri ve Native İletişim



Yalnızca native platformlar (web'de yok)

```dart
import 'dart:io';

final file = File('notlar.txt');
final text = await file.readAsString();
```

Büyük dosyalar için belleği tıkamamak adına asenkron parça okuması tercih edilmelidir:

`openRead()` + `LineSplitter`

Gemini Notebook

Ağ İletişimi — HTTP İstemci ve Sunucu Mimarisi

İki farklı cihazı birbirine bağlayan İstek (Request) ve Yanıt (Response) modeli.

İstemci tarafı

`package:http` gibi üst düzey paketler.

İşlev: Dış dünyaya İstek (Request) gönderir.

İstek (Request)



Yanıt (Response)

Sunucu tarafı

`dart:io` ile HTTP sunucusu (`HttpServer`).

İşlev: Gelen isteği işler ve Yanıt (Response) döndürür.

Gemini Notebook

JavaScript Interop — Web Platformu Köprüsü

Dart ve JavaScript'in birlikte çalışması, web API'lerine kesintisiz erişim.

Güncel Yaklaşım	Eski Yaklaşım
Kütüphaneler: `dart:js_interop` + `package:web`	Kütüphaneler: `dart:html`, `dart:svg`
Özellik: WebAssembly (Wasm) derlemesini destekler.	Özellik: Wasm desteklemez.
Durum: **Tavsiye Edilen (Modern)**	Durum: **Dart 3.7 ile kullanımdan kaldırıldı.**

Gemini Notebook

Birleşik Veri Boru Hattı (Pipeline)

İzole kütüphanelerin tek bir veri akışında mükemmel uyumu.



---

kütüphane seçimi | koleksiyonlar | dönüşümler | Future | Stream | hata yönetimi

Gemini Notebook

Dart resmî dokümantasyonu: `https://dart.dev`

Dart API referansı: `https://api.dart.dev`

Sorular?

Gemini Notebook